

Spring Framework

Features of Spring framework:

IoC inversion of control container: in a normal condition or without spring framework we would need to create an instance(object) of the class and inject it where it should, manage it. This creates tight coupling. However the inversion of control allows spring boot using @Autowired to create an object for us, inject where its needed.

MVC: Model, View, control architecture is supported to separate data, http requests and the app logic.

Data access framework: it helps simplify and organize database connections.

Aspect Oriented Programming: AOP separates cross-cutting concerns from business logic.

A java bean is an object managed its lifecycle by spring container. The container creates and manages the bean. This bean can be wired using spring xml configuration, or spring boot annotations.

@configuration to configure a class must be non-final and public

@Bean to configure a bean and it must be not private and not final

A @Component is a class that becomes a Spring-managed bean directly, while a @Configuration class is also a bean but primarily serves as a factory for defining other beans.

@Autowired actually means

“Spring, resolve and inject the required bean(s) here.”

It can be applied to:

- Constructors, Fields, Setter methods

In the app configuration class when we define beans using methods we annotate those methods with @Bean, and this bean that is returned from the method is named the same as the method name but if we wrote @Bean(“name”) now the spring boot will name that bean as name.

Dependency injection mostly means bean injection.

Constructor injection is when the dependency or the java spring bean is passed into another bean or class's constructor

`@Qualifier("primary")` is a way that we use to prioritize or name our beans in case we have 2 beans of the same type, sometimes we would need to `@Autowired` them but our spring container does not know which one, so qualifier is how we tell the spring container which one it is.

Field injection is declaring a dependency or another bean of java spring and declaring it into another, directly (very discouraged since it makes the testing harder)

method injection in this case it to declare a method inside our class and inside this method we make an instance of our dependencies and `@autowired` it.

Setter injection works the same but the method is called set, like normal setters in normal OOP.

The bean scope is when is it available during the project lifecycle to interact with other beans, and that is by default is a singleton: only one instance of the bean.

Bean scopes: Singleton Prototype Request, Session, Application, Web socket.

`@Scope("Prototype")` or any other scope to help us define the scope for the bean. After `@Bean`

Environment abstraction bean environment,

`@Profile` is a conditional switch that determines whether a bean is loaded into the Spring container based on the active environment. Whether its testing, development or production

`@Value("Actual value")` is used to inject value into a variable, whether its hardcoded or comes from another environment or class. Or class path, or file or variable.

Some best practices for spring boot:

Split configuration classes based on the architecture. Into multiple classes

Spring boot in a nutshell is a less configuration spring.

Spring Boot simplifies Spring by making applications **self-contained, auto-configured, and easy to run**, mainly through standalone execution, embedded servers, and starter dependencies.

Spring REST

Representational stateless transfer, a software architecture to make web services to which usually are called RESTFUL APIs.

The main idea is to treat the network components as objects that can be accessed using standard http methods

It's a client server architecture they can work together but each one of them works and handles data independently.

Its stateless meaning the server does not store the requests only executes them

Its layered cacheable system

we have linking of resources

we have standard methods **GET, POST, PUT, DELETE**

the resources that hold the data should be plural nouns for instance

GET / accounts → retrieves all the accounts

GET/ accounts/1 → retrieves the account with the id 1

In case of nested resources payments inside the accounts it should be like this

GET/accounts/1/payments/2

This returns the payment of id 2 inside the account of id 1

We use http messages **GET, POST, PUT, DELETE, PATCH, OPTIONS, HEAD** to specify what should be done

GET method

```
@GetMapping("/users/{id}")
```

```
Public void printProduct(){}
```

This annotation is triggered aka mapped into the function its on once the http request of get/users/{id} is accessed it just fetches data from the server and show it to the user its very safe

POST

```
@PostMapping("/users")
```

Post is used to create a new resource in the database

PUT

```
@PutMapping
```

```
@PutMapping("/users/{id}")
```

```
public User updateUser(@PathVariable Long id, @RequestBody User user) {  
    user.setId(id);  
    return userService.update(user);  
}
```

Update an existing resource by replacing it completely.

How it behaves:

- Client sends full updated object
- Existing data is completely replaced
- Missing fields may be overwritten (often set to null)

DELETE

```
@DeleteMapping("/users/{id}")
```

```
public void deleteUser(@PathVariable Long id) {  
    userService.delete(id);  
}
```

Is used to remove a resource from the database

PUT

```
@PutMapping("/users")
```

```
@PatchMapping("/users/{id}")
```

```
public User patchUser(@PathVariable Long id, @RequestBody Map<String, Object> updates) {  
    return userService.partialUpdate(id, updates);  
}
```

Update only specific fields of a resource (not the whole object).

Real-life analogy:

Changing just your email address, not your entire profile.

OPTIONS — Discover Supported Methods

```
@RequestMapping(value = "/users", method = RequestMethod.OPTIONS)
```

```
public ResponseEntity<?> options() {  
    return ResponseEntity.ok()  
        .header("Allow", "GET, POST, PUT, DELETE, PATCH, OPTIONS")  
        .build();  
}
```

Purpose:

Ask the server what operations are allowed on a resource.

HEAD — Retrieve Headers Only

```
@RequestMapping(value = "/users/{id}", method = RequestMethod.HEAD)
```

```
public ResponseEntity<Void> headUser(@PathVariable Long id) {  
    if (userService.exists(id)) {  
        return ResponseEntity.ok().build();  
    } else {  
        return ResponseEntity.notFound().build();  
    }  
}
```

Purpose:

Same as GET, but returns only headers (no response body).

Response Status code

First digit defines the class of the response while the last 2 digits they don't have any specific

1xx: Informational

2xx: Success.... 200 OK -> standard response for success, 201 CREATED -> A new resource was created from the post method, 204 no content DELETE PUT POST

3xx: Redirection, 304 NOT Modified → for caching, if the current version matches the version in the client cache we get this message from the server and no more content. We just use the local version of the request.

4xx: client error, 400 bad request, the server was not able to understand the request because of invalid syntax. 401 unauthorized -> means requires user authentication. 403 forbidden the client does not have the necessary authorization for the certain resources.

5xx: server error, 500 server error when the server has an undefined issue and no more information is known or provided. 503 server unavailable, which means that the server is down because its overloaded or under maintenance.

What is a @RestController

A `@RestController` is a Spring annotation used to define a controller that handles HTTP requests and returns data directly in the response body. A controller processes incoming HTTP requests, executes application logic, and produces a response. Unlike a regular `@Controller`, which typically returns a view (HTML page), a `@RestController` returns raw data such as JSON or XML.

@ResponseStatus

What is `@ResponseStatus`?

`@ResponseStatus` is an annotation used to:

Manually set the HTTP status code of a response

```
@PostMapping("/users")
@ResponseStatus(HttpStatus.CREATED)
public User createUser(@RequestBody User user) {
    return user;
}
```

@RequestBody

Take the HTTP request body (usually JSON) and convert it into a Java object. `@RequestBody` is an annotation used in Spring MVC / Spring Boot to indicate:

Take the raw HTTP request body, deserialize it (usually from JSON), and bind it to a Java object so it can be used in backend logic

Example

```
@PostMapping("/user/post")
@ResponseStatus(HttpStatus.CREATED)
@Autowired → incorrect there is no dependency injection to be done
public String printUser( @RequestBody Order order){
    return "welcome the order details are " + order.toString();
}
```

To explain the dynamics, here we have a post (create new user or data), and we just take the data from the user and printing it no logic is happening in the backend, the `@RequestBody` is the focus. The request body tells the Spring boot that we will be passing data in the http request body and this data should be changed from JSON into JAVA object called order

After collecting this data we should then inside the order class map our json into java variables.

Without @RequestBody spring boot will try to deserialize the data automatically

To map the data into java variables we should use @JsonProperty if the variable names are different from the JSON names of the corresponding variable

Record class

Records are final by default and all their fields are also final

A concise way to declare a class whose main purpose is to hold data, with built-in immutability and automatically generated methods.

Example from the online course: when we were mapping the JSON data from the request body into the class we were using a class called order, inside this class we mapped the json variables on the variables of the class, then declared setters and getters.

A record class declared as public record order(variable1, variable2, variable3){}

The record creates setters and getters and toString() automatically and maps the json data on our record.

A record can implement an interface but cannot extend a class.

```
Public record OrderRequest(  
    @JsonProperty("customer_name") String customerName,  
    @JsonProperty("product_name") String productName,  
    int quantity  
) {}
```

@pathvariable

@PathVariable is used to capture dynamic values from the URL path and pass them into a controller method.

```
@GetMapping("/orders/{id}")  
  
public String getOrder(@PathVariable int id) {  
    return "Order ID: " + id;}  
}
```

@RequestParam

Variable as request parameter

Request dispatching

```
@GetMapping("/orders")
```

```
public String getOrder(
```

```
    @RequestParam int id,
```

```
    @RequestParam String status
```

```
) {
```

```
    return id + " " + status;
```

```
}
```

A method name should be unique even if it accepts different path variable or parameters still it should hold a unique name, for each method so for example

We can have 2 method called /hello but for different methods so one can be get and the other post

But we cannot have 2 hello methods even if the first one is accepting path variable and the second not since both are GET

Data source configuration

Spring:

Datasource:

url:

username:

password:

driver-class-name: for this we need to add the database dependency of mysql

Persistent entity for the database

In the context of Java Spring Boot and Spring Data JPA, a persistent entity is a central concept that represents how Java objects are mapped to database tables.

Where each java object instance is stored as row in this persistent entity

Database row → loaded back as a Java object

JPA defines the rules for mapping Java objects to relational database tables.

Hibernate (usually used under Spring Boot) is the engine that performs the actual work.

Spring Data JPA simplifies repository access (CRUD operations) so you don't write SQL for most operations.

@Entity

Is how you define a java class to be a persistent JPA entity

@Table(name = "table_name") this step is optional, to give a name for the table.

@Column (optional)

Used to customize column details:

@Column(name = "email_address", nullable = false)

private String email;

@Column(

name = "email_address",

nullable = false,

unique = true,

length = 150

) all of the information here is optional

```
@Table(  
    name = "users",  
    uniqueConstraints = {  
        @UniqueConstraint(columnNames = {"email", "tenant_id"})  
    }  
)
```

this is very strong and it forces the table to have only one email and one tenant_id per user name

`@Entity` `@Table` we have to have an empty constructor—we need to specify a primary key each time we create an entity

`@Id`

is one of the most fundamental annotations. It defines the primary key of a persistent entity.

Without it, JPA cannot uniquely identify rows in the database, and the entity is not valid for persistence.

`@GeneratedValue`

Without `@GeneratedValue`, you must assign IDs manually:

```
user.setId(1L);
```

but on using `@GeneratedValue` the Spring JPA assigns a value to our primary key, mostly surrogate key

`@GeneratedValue(strategy = GenerationType.X)`

Here we can get more specific about how the automatic values will be assigned to our primary key. The best so far is AUTO

.IDENTITY is the equivalent of using AUTO_INCREMENT in SQL

Jpa:

 Hibernate:

 Ddl-auto: create

 Show-sql: true

 Properties:

 Hibernate:

 Format-sql: true

 Database-platform

jpaRepository Interface

The hierarchy is:

Repository (marker interface)

↓

CrudRepository

↓

PagingAndSortingRepository

↓

JpaRepository

JpaRepository is the **most feature-complete interface in this chain**. It will give us more control, over our repo and the most optioned layer. It a generic interface.

It has all the CRUD operations: **CrudRepository**

Adds basic CRUD:

save()

findById()

findAll()

delete()

count()

existsById()

flush()

saveAndFlush()

`deleteInBatch()`

`findAll(Specification)`

`userRepository.save(user);`

what happens here is that JpaRepository checks whether the user entity object has an already Id if `id == null` then it creates a row using `persist()`, if its untracked and exists meaning that `id != null`, it calls for `merge()`. If it exists and is tracked nothing happens.

`Optional<User> user = userRepository.findById(1L);` is equivalent to the sql of

`SELECT * FROM TABLE WHERE USER_ID = 1`

`@query`

To get started with spring jpa we should create an **interface** of our class that holds the table example:

`public interface StudentInterface extends JpaRepository <Student, Integer>`

here the Student is the class name, and the integer is the Id which is primary key

no annotation needed here for the student jpa interface

to use our interface we should inject it in some component, for this example we inject it in our controller

to add to our repository we should use `repository.save(object)` object here is an equivalent of a row

to find we can use `findById`

or `findAll`

to add a new method we can define it in our interface of the repository and the java jpa will create a query for us

Here's the simple rule for capitalization across your entire Spring Boot application: Java field names start with a lowercase letter (e.g., fName, email, id), and their corresponding getter/setter methods capitalize the field name after get/set (e.g., getFName(), setFName()) — but Spring Data JPA repository methods must match the exact field name starting with lowercase (findByFNameContaining, NOT findByFNameContaining). Database columns via @Column(name = "f_name") can be whatever naming convention you want (usually snake_case). Path variables in @GetMapping like /{name} are independent and just pass values. The only places where capitalization breaks everything are: (1) repository method names must match Java field names exactly (case-sensitive), (2) getters/setters must follow the getFieldname() pattern with capital after get, and (3) JSON responses rely on those getters being correct — everything else (URLs, database columns) is more flexible but keeping consistent lowercase for fields and camelCase for getters avoids confusion.

@OneToOne(cascade = CascadeType.ALL, mappedBy = "class_name") this means whenever our entity object is deleted the object which its annotated on will also be deleted.

Defining a foreign key:

```
public class Student {  
  
    private Locker locker; // ← This field name is "locker"  
  
    @OneToOne  
  
    @JoinColumn(name = "locker_id") // ← Database column name to be created in student  
    entity, with the naming of the column  
  
    private Locker locker; // ← Same field!  
  
}  
  
// Locker.java  
  
public class Locker {  
  
    @OneToOne(mappedBy = "locker") // ← Points to Student's field name "locker" so this is  
    where the foreign key to be found, meaning in this class the foreign key comes from the locker  
  
    private Student student;  
  
}
```

mappedBy goes on the **non-owning** side and tells JPA: "Look in the other class's locker field to find the foreign key."

@JoinColumn(name = "locker_id") means:

"Create a column called locker_id in THIS table. This column will hold the ID of a row from the OTHER table (Locker)."

mappedBy Explained Simply:

mappedBy tells JPA: "I'm not the boss. Go look in the other class to find how these connect."

java

```
@OneToMany(mappedBy = "category") // Look in Product.class for a field called "category"
```

```
private List<Product> products;
```

CascadeType.PERSIST

Save mother → children also saved

CascadeType.REMOVE

Delete mother → children also deleted

CascadeType.ALL

Everything applies to children

Example:

java

```
@Entity
```

```
public class Category {
```

```
    @OneToMany(mappedBy = "category", cascade = CascadeType.ALL)
```

```
    private List<Product> products = new ArrayList<>();
```

```
}
```

Now when you do:

java

```
Category electronics = new Category();
```

```
Product laptop = new Product();
```

```
laptop.setCategory(electronics);
```

```
electronics.getProducts().add(laptop);
```

```
categoryRepository.save(electronics); // BOTH category AND product saved automatically!
```

@JSONBACKREFERENCE

Service → it separates the logic from the data layer (entity) and representation layer (the controller) the service layer is responsible for implementing the business logic and hard calculations

And by this separation using the service layer we can focus on testing the business logic solely

DTO (Data Transfer Object)

The main purpose is to encapsulate data that needs to be transferred among systems

Usually includes attributes, we need to include a mechanism between the entities and the outside world to prohibit the exposure of certain data inside the entity/database. → mapper

The mapper allows us to have multiple different representations of the same entity.

DTO helps with data separation, means that we can change the logic and it won't affect the end user, we can hide some confidential data

It improves also the performance by sending minimal data to the server, decreasing the time.

For DTOs we use records instead of classes

Cascade types and when to use what

Type	What it does	When to use
PERSIST	Save child when parent is saved	Child can't exist without parent
MERGE	Update child when parent is updated	Changes to parent should sync to child
REMOVE	Delete child when parent is deleted	Child has no meaning without parent
REFRESH	Reload child from DB when parent is reloaded	Keep child in sync with parent
DETACH	Remove child from persistence context when parent is detached	Rarely used directly
ALL	All of the above	Strong parent-child where child has no independent existence

Best approaches to organize classes, records, and interfaces as the application grows.

By feature approach, each of the packages contains all the classes all the MVC. (favored)

By the layer we separate the architecture to MVC and other configuration

By domain the system is packaged according to the domains of the business logic, so for a health care system as an example we would have. Patient management, billing, scheduling, and medical records each in a package

Last approach is to break it into features and inside each feature we break the classes into MVC layers

Data Validation in REST API

Ensures data integrity, consistency and reliability of our data. It prevents attacks XSS and SQL injection reduction. And error preventing

For validation in spring boot we have spring-boot-starter-validation dependency

We can validate objects using annotations

We need to add the validation on the first object on our project for this example it was the DTO

`@NotEmpty(message = "first name should not be empty hommie")`

`@Valid`

`@ExceptionHandler(exceptionclass.class)`

Error exception handling happens in the controller

`@AssertTrue` → used for Boolean fields

`@AssertFalse`

`@Digit(range-range)`

`@Email`

`@Future` for event date

Testing:

Test help with the quality assurance

Regression testing

Test can serve as a documentation and use cases and

Spring helps us in testing by providing some annotations

@SpringBootTest

For the testing phase, testing A DTO is not encourage or in other words does not make any sense as long as the DTO does not hold any logic. unlike the Mapper

when we open a class we can hold ctrl + shift + T to make IntelliJ refer us to the class. in case we don't have a test class for the current class we can just right click generate then test.

@BeforeEach

@AfterEach

@BeforeAll

@AfterAll

while testing in the dev environment what we can do is we can insert some data into the database using before all and after all we can delete all of it using the @BeforeAll and @AfterAll

assertNotNull()

assertEquals

assertThrows(NullPointerException.class, () -> mapper.toStudent(dto:null))

//given

//when → during the when we need to create a test case for when the student dto is nul

//then

Test isolation means that we need to run a test on a certain component in isolation of its dependencies → for this we use Mockito

we mock the dependencies using @Mock when we declare them in the testing class

we need to mock the calls, of which our class calls in the dependencies

Mockito.when()

verify(times, the thing needs to be counted)

In Normal Junit testing, we rarely make the instance of the class undertesting a public static void. since it will be shared across all tests which is dangerous. for this purpose we usually use @BeforeEach to create a new instance before each test

when deciding on what to test we look at a class, we identify what input can it take, also using extreme cases, what output does it produce, what type of exception can happen.

the Given I still need to figure how to get it, the when is the functionality at work, the then is the assertion of comparing the expected output to the expected.

in each mock test we need to have one real actual code to compare the mocked results to it

when mocks the behaviour of the real thing. meaning when deciding on the whens what we need is to look for the method calling in our mocked function, and return what it returns the same data type that is returned in the actual code

```
verify(studentMapper, times(1)).toStudent(inputDto);
```

Translation:

"Check that studentMapper.toStudent() was called **exactly once** with **exactly this inputDto**"

Some Spring JPA Notes

Lombok Annotations and what uses we have for them:

**all Lombok annotations produce the same byte code as writing the manual code, so its better for readability*

@Getter → is a Lombok annotation on a class level to generate getters for all variables inside the class

@Setter

@AllArgsConstructor

@NoArgsConstructor

@Data includes many annotations like: @Getter @Setter @RequiredArgsConstructor @ToString @EqualsAndHashCode

so a no args constructor is still needed

difference between hibernate and spring data jpa:

JPA = Interface (the "contract" or "rules")

Hibernate = One implementation of that interface

@Builder annotation use cases:

Builder is a Lombok annotation that is used to create objects in java in a clean way, its best used with DTOs or when the args are optional; without the builder we have to include all the arguments needed in the constructor but with builder we can skip some of them and they will stay null or fall back to the default values.

@Column annotation use cases and parameters:

is a JPA annotation that maps a Java class field to a database column. It allows you to customize the column definition and constraints.

for this example the name = user_name is only the column name when we deal with the email column in our project its still email, getEmail and so on .

Parameter	Purpose	Default	Example
name	Database column name	Field name	name = "user_email"
unique	Enforce unique constraint	false	unique = true
nullable	Allow NULL values	true	nullable = false
length	Column size (for strings)	255	length = 50
columnDefinition	Full SQL definition	""	columnDefinition = "TEXT"
table	Specific table for this column	Main table	table = "user_details"
insertable	Include in INSERT statements	true	insertable = false
updatable	Include in UPDATE statements	true	updatable = false
precision	Total digits (for decimals)	0	precision = 10
scale	Decimal places (for decimals)	0	scale = 2

for the createdAt; @Column(insertable = false, updatable = false) any thing that is totally managed by the DBMS is false as insertable and false as updatable

@Table annotation explanation and use cases:

table annotation is class level or entity level where we can force some constraints on it, some of them are to set a name, to set some constraints like unique constraints.

and one of the most important is to create an index from this table

when configuring our database connection inside the .yml or .properties in mysql we can pass this parameter at the end of the url to create a database if its not made.

?CreateDatabaseIfNotExist=true

The id auto-generation strategies:

while I believed that .IDENTITY is the safest most efficient it turns out that im partially wrong, the strategy is very safe however its not the most efficient ist actually very slow and for this we shall substitute it for .Sequence

now we are going to explain the sequence and start using it.

unfortunately does not work for mySQL so later we might need to migrate the project to Postgresql

CommandLineRunner Class:

is an interface that runs code after the Spring context is loaded but before the application starts accepting requests.

the many to many relationship

in many to many relationship in JPA one of the 2 entities is considered to be the owner of the relationship the other is considered the inverse and has no foreign keys

the inverse the (non-owning side) should have the mapped by attribute

on the owning side we should have @JoinTable(name = "new column name", joinColumns ={@JoinColumn = owner_id

@JoinColumn =

}, inverseJoinColumn= @JoinColumn = "inverse_id")

in one to one relationships we should define the @JoinColumn on both sides

composition over inheritance, in general composition is more flexible and favored more.

@MappedSuperclass

@SuperBuilder

we can move all the common fields into that base entity

in case we migrated the primary key of an entity to the base entity we should extend the base entity inherit it by the child class

@EqualsAndHashCode(callSuper = true)

@Discriminator

@Inheritance

in case of inheritance or super, child class as entity. the child class columns will appear in the parent class table

@DiscriminatorColumn

@DiscriminatorValue

for the example in the lecture the instructor created a JPA entity, that's called resources and it has 3 children, text, File, and video the discriminatorcolumn was set to be called dicr_type

and the value was set in each child in the same order T, F, V so when we insert a resources row with the value for file the discriminator will be F

@Inheritance(strategy = single table) → this is the strategy that was used to make each child joined on the parent table once we set it to

@Inheritance(strategy = joined) → each child table will have its own entity with a foreign key referring to the parent class,, when using this strategy no discriminator column is needed in the parent class nor a discriminator value for the children entities

on using strategy = join, the spring hibernate will create an automatic entities with the joincolumn already done no need to do manual onetoone @JoinColumn

@PrimaryKeyJoinColumn

Abstract super classes

@Inheritance(strategy = table per class)

this does the opposite of the single table, it maps the fields of the super class to the sub class, and gives the sub class the primary key as a foreign key . using this inheritance strategy we cannot use identity id generation method and if we query the super class we will get the information of the subclasses as well. which is going to happen using a union

@Ploymorphism(explicit) → this annotation is used on the sub class entity so its excluded when the super class entity is queried, its implicit by default.

Embedded ID and embedded Entities → it's the strategy of using multiple primary keys to identify a row.

@Embeddable

@EmbeddedId

for this example we created a class called OrderId which will hold the composite primary key, for it to work it should have the annotation for @Data, @Embeddable, and it should implements serializable

on the other hand the entity that will have this composite key instead of having a primary key or @Id it will have @EmbeddedId over a variable that is called OrderId

@Embedded helps us reuse the entity in other entities not just one time but multiple times, for example if we have to include the address in multiple different entities, we might need to include the address in two different entities so we may declare a class of type data and give it the annotation @Embeddable on the class address and then declare that address class anywhere using @embedded to include those fields in the class that we need to embed to

```
List<Author> findByFirstNameAAndLastName(String FirstName, String LastName);
```

```
Optional<Author> findByFirstNameAAndLastName(String FirstName, String LastName);
```

```
List<Author> findByFirstNameIgnoreCase (String FirstName);
```

```
List<Author> findByFirstNameContainingIgnoreCase(String FirstName);
```

```
List<Author> findByFirstNameStartsWithIgnoreCase(String FirstName);
```

```
List<Author> findByFirstNameStartsInIgnoreCase(List<String> FirstNames);
```

Java faker a dependency that can generate for us some fake data to work with.

if we save an entity using repository.save(entity) but we specify the id we will be updating that entity not inserting it, and we have to make sure that this id actually exists

@Modifying

@Transactional

@Query("sql query") → lets us run a query on our system and is used for update, we should declare it and use it in the repository interface

@Transactional on update and/or delete query

@NamedQueries → reusable, optimize the performance for frequently used queries

Spring data specification interface

List<ClubEntity> findByAddress_ZipCodeAndSports_Sport_SportsCategory(Integer zipCode, String category);

Address Navigate to the address field on ClubEntity. That field is of type AddressEntity, so Spring knows to JOIN the addresses table.

_ Go one level deeper — we are now inside AddressEntity.

ZipCode The field we want to filter by inside AddressEntity. It is an Integer, a simple value, so we stop navigating here. This becomes the first WHERE condition.

And A Spring Data JPA keyword meaning combine two conditions with SQL AND. Everything before it is condition one, everything after it is condition two.

Sports Navigate to the sports field on ClubEntity. That field is of type List<SportClubEntity>, so Spring knows to JOIN the sports_club table.

_ Go one level deeper — we are now inside SportClubEntity.

Sport Navigate to the sport field on SportClubEntity. That field is of type SportEntity, so Spring knows to JOIN the sports table.

_ Go one level deeper — we are now inside SportEntity.

SportsCategory The field we want to filter by inside SportEntity. It is a String, a simple value, so we stop navigating here. This becomes the second WHERE condition.

(Integer zipCode, String category) The two parameters Spring maps to the two conditions in order. zipCode feeds into ZipCode, category feeds into SportsCategory.